

Module 2

Detailed Notes

Chapter 3 — Console I/O

Topics Covered

- Introduction to Console I/O
- Reading and Writing Characters
 - getchar() and putchar()
 - Problem with getchar() and Alternatives: getch(), getche()
- Reading and Writing Strings
 - gets() and puts()
- Formatted Console I/O
- printf()
 - Format Specifiers
 - Printing Characters, Numbers, Addresses
 - The %n Specifier
 - Format Modifiers: Minimum Field Width, Precision, Justification
 - Handling Other Data Types (l, h, L, hh, ll modifiers)
 - The * and # Modifiers
- scanf()
 - Format Specifiers
 - Inputting Numbers, Characters, Strings, Addresses
 - Using a Scanset %[]
 - Format Modifiers
 - Discarding White Space and Non-White-Space Characters
 - Suppressing Input with *
 - Passing Addresses to scanf()

CHAPTER 3 — CONSOLE I/O

The C language does not define any keywords that perform I/O. Instead, input and output are accomplished through library functions. C's I/O system is an elegant piece of engineering that offers a flexible yet cohesive mechanism for transferring data between devices.

Key Concept	Detail
Header file required	<stdio.h> — must be included for all standard I/O functions
Console vs File I/O	Technically little distinction, but conceptually in different worlds; this chapter covers console I/O
Standard input / output	Console functions operate on standard input (keyboard) and standard output (screen); these can be redirected to other devices
TTY-based output	Standard C defines only TTY-based (text terminal) console functions; screen control, graphics, and window functions are compiler extensions
I/O operators (C++)	C++ adds I/O operators (<<, >>); these are NOT supported by C

NOTE *Standard C does not define functions that perform screen control (cursor positioning), display*

graphics, or write to Windows dialog boxes. Most compilers add these as extensions.

READING AND WRITING CHARACTERS

The simplest console I/O functions are `getchar()`, which reads a character from the keyboard, and `putchar()`, which writes a character to the screen.

`getchar()` and `putchar()`

Prototypes:

```
int getchar(void);  
int putchar(int c);
```

Function	Description	Return Value
<code>getchar()</code>	Waits until a key is pressed and returns its value. The keypress is automatically echoed to the screen. Declared as returning <code>int</code> , but can be assigned to a <code>char</code> variable (character is in the low-order byte; high-order byte is usually zero).	Character read, or EOF on error
<code>putchar()</code>	Writes a character to the screen at the current cursor position. Declared as taking an <code>int</code> parameter, but normally called	Character written, or EOF on error

Function	Description	Return Value
	with a char argument. Only the low-order byte of the parameter is output.	

NOTE *EOF is a macro defined in <stdio.h>, often equal to -1 . It is returned when an input function tries to read past end-of-file, or when an error occurs.*

Example — reads characters and displays them in reverse case (uppercase↔lowercase); stops at a period:

```
#include <stdio.h>
#include <ctype.h>
int main(void)
{
    char ch;
    printf("Enter some text (type a period
to quit).\n");
    do {
        ch = getchar();
        if(islower(ch)) ch = toupper(ch);
        else ch = tolower(ch);
        putchar(ch);
    } while(ch != '.');
    return 0;
}
```

A Problem with `getchar()`

For many compilers, `getchar()` is implemented with line-buffered input — it buffers input until ENTER is pressed before returning any character. This causes two issues:

- You must press ENTER before any character is returned
- Only one character is returned per call, but line buffering may leave one or more characters waiting in the input queue — annoying in interactive environments

NOTE *Although it is permissible for `getchar()` to be implemented as an interactive function, it seldom is.*

Alternatives to `getchar()` — `getch()` and `getche()`

Since `getchar()` may not be useful in interactive environments, most C compilers provide two alternative non-standard functions:

```
int getch(void);    /* prototype in  
<conio.h> */  
int getche(void);   /* prototype in  
<conio.h> */
```

Function	Echo?	Waits for ENTER?	Standard C?	Header
<code>getchar()</code>	Yes	Yes (line-buffered)	Yes	<code><stdio.h></code>
<code>getch()</code>	No	No — returns immediately	No (common extension)	<code><conio.h></code>

Function	Echo?	Waits for ENTER?	Standard C?	Header
getche()	Yes	No — returns immediately	No (common extension)	<conio.h>

- For some compilers (e.g., Microsoft Visual C++), these functions have a leading underscore: `_getch()` and `_getche()`
- `getch()` and `getche()` are frequently used instead of `getchar()` when a character needs to be read interactively

Example — same reverse-case program using `getch()` for immediate, non-buffered input:

```
#include <stdio.h>
#include <conio.h>
#include <ctype.h>
int main(void)
{
    char ch;
    printf("Enter some text (type a period
to quit).\n");
    do {
        ch = getch();
        if(islower(ch)) ch = toupper(ch);
        else ch = tolower(ch);
        putchar(ch);
    } while(ch != '.');
    return 0;
}
```

```
/* Each keypress is immediately transmitted
and displayed in reverse case.
   Input is no longer line buffered. */
```

NOTE *When using Microsoft's Visual C++ compiler, `_getche()` and `_getch()` are not compatible with standard C input functions like `scanf()` or `gets()`. Special versions such as `cscanf()` or `cgets()` must be used instead.*

READING AND WRITING STRINGS

The next level of console I/O in terms of complexity and power are `gets()` and `puts()`, which enable reading and writing strings of characters.

`gets()`

The `gets()` function reads a string of characters entered at the keyboard and stores them at the address pointed to by its argument.

Prototype:

```
char *gets(char *str);
```

- `str` is a pointer to a character array that receives the characters entered by the user
- You can type characters until you press ENTER (carriage return)

- The carriage return does NOT become part of the string — a null terminator '\0' is placed at the end instead
- gets() cannot return a carriage return (but getchar() can)
- You can correct typing mistakes using the BACKSPACE key before pressing ENTER
- gets() returns str (same pointer that was passed in)

Example — reads a string and prints its length:

```
#include <stdio.h>
#include <string.h>
int main(void)
{
    char str[80];
    gets(str);
    printf("Length is %d", strlen(str));
    return 0;
}
```

NOTE *gets() performs NO boundary checking on the receiving array. The user could enter more characters than the array can hold, causing a buffer overflow. Use gets() only for sample programs and simple utilities. For commercial code, use fgets() instead (described in Chapter 9), which limits the number of characters read.*

puts()

The puts() function writes its string argument to the screen followed by a newline character.

Prototype:

```
int puts(const char *str);
```

- Recognizes the same backslash escape sequences as `printf()` — e.g., `\t` for tab
- Requires far less overhead than `printf()` because it can only output a string — it cannot output numbers or perform format conversions
- Therefore, `puts()` takes up less space and runs faster than `printf()`
- Used when no format conversions are required
- Returns a nonnegative value if successful, EOF if an error occurs
- When writing to the console, the return value is seldom monitored since errors rarely occur

```
puts("hello"); /* displays: hello  
(followed by newline) */
```

Basic Console I/O Functions Summary (Table 8-1)

Function	Operation	Standard C?
<code>getchar()</code>	Reads a character from keyboard; usually waits for carriage return (line-buffered)	Yes

Function	Operation	Standard C?
<code>getche()</code>	Reads a character with echo; does not wait for carriage return; returns immediately	No (common extension)
<code>getch()</code>	Reads a character without echo; does not wait for carriage return; returns immediately	No (common extension)
<code>putchar()</code>	Writes a single character to the screen	Yes
<code>gets()</code>	Reads a string from the keyboard until ENTER is pressed	Yes
<code>puts()</code>	Writes a string to the screen followed by a newline	Yes

FORMATTED CONSOLE I/O

The functions `printf()` and `scanf()` perform formatted output and input — they can read and write data in various formats under your control.

Function	Direction	Purpose
<code>printf()</code>	Output (keyboard → screen)	Writes formatted data to the console; can output any built-in data type and null-terminated strings
<code>scanf()</code>	Input (keyboard)	Reads formatted data from the keyboard; can read all built-in

Function	Direction	Purpose
	→ variables)	data types and automatically converts numbers to proper internal format

printf()

Prototype:

```
int printf(const char *control_string,  
...);
```

- Returns the number of characters written, or a negative value if an error occurs
- The control_string consists of two types of items:
 - 1. Characters to be printed literally on the screen
 - 2. Format specifiers that define how subsequent arguments are displayed
- A format specifier begins with a % sign and is followed by a format code
- There must be exactly the same number of arguments as format specifiers
- Format specifiers and arguments are matched in order from left to right

Example:

```
printf("I like %c %s", 'C', "very much!");  
/* Output: I like C very much! */
```

```
/* %c matches 'C', %s matches "very much!"  
*/
```

printf() Format Specifiers (Table 8-2)

Code	Format / Description
%c	Single character
%d	Signed decimal integer
%i	Signed decimal integer (same as %d; supported for historical reasons)
%u	Unsigned decimal integer
%f	Decimal floating-point (argument must be type double)
%e	Scientific notation with lowercase e — e.g., 1.23e+04
%E	Scientific notation with uppercase E — e.g., 1.23E+04
%g	Uses %e or %f, whichever produces shorter output
%G	Uses %E or %F, whichever produces shorter output
%o	Unsigned octal integer
%x	Unsigned hexadecimal integer (lowercase letters a–f)
%X	Unsigned hexadecimal integer (uppercase letters A–F)
%s	String of characters (null-terminated)

Code	Format / Description
%p	Displays a pointer (memory address in architecture-compatible format)
%n	Stores the number of characters written so far into the integer pointed to by the corresponding argument (does not print anything)
%%	Prints a literal % sign
%a	Hexadecimal floating-point output in form 0xh.hhhhp+d (C99 only)
%A	Hexadecimal floating-point output in form 0Xh.hhhhP+d (C99 only)

Printing Characters and Strings

Use **%c** to print a single character — the matching argument is output unmodified.

Use **%s** to print a null-terminated string.

```
printf("%c", 'A');          /* prints: A
*/
printf("%s", "hello");      /* prints:
hello */
```

Printing Numbers

- **%d** or **%i** — signed decimal integer (equivalent; both supported for historical reasons and `scanf()` compatibility)
- **%u** — unsigned integer
- **%f** — floating-point in decimal format (argument must be double)

- %e or %E — scientific notation; use %E for uppercase E
- %g or %G — selects %f or %e (whichever is shorter); use %G for uppercase E
- %o — unsigned octal
- %x or %X — unsigned hexadecimal; %x uses lowercase a–f, %X uses uppercase A–F

Example — %g selects shorter of %f or %e:

```
#include <stdio.h>
int main(void)
{
    double f;
    for(f=1.0; f<1.0e+10; f=f*10)
        printf("%g ", f);
    return 0;
}
/* Output: 1 10 100 1000 10000 100000
1e+006 1e+007 1e+008 1e+009 */
```

Example — octal and hexadecimal output using %o, %x, %X:

```
for(num=0; num < 16; num++) {
    printf("%o ", num);    /* octal */
    printf("%x ", num);    /* hex lowercase */
    printf("%X\n", num);   /* hex uppercase */
}
```

Displaying an Address

Use `%p` to display a memory address. `printf()` will display the address in a format compatible with the addressing used by the computer.

```
#include <stdio.h>
int sample;
int main(void)
{
    printf("%p", &sample); /* displays the
address of sample */
    return 0;
}
```

The `%n` Specifier

The `%n` specifier is different from all other format codes — instead of displaying something, it causes `printf()` to store the number of characters output so far into the integer variable pointed to by its corresponding argument.

- The value corresponding to `%n` must be a pointer to an integer variable
- After the call to `printf()` returns, this variable will contain the number of characters output up to the point where `%n` was encountered
- Used primarily to enable dynamic formatting

```
#include <stdio.h>
int main(void)
{
    int count;
    printf("this%n is a test\n", &count);
}
```

```
    printf("%d", count);  
    return 0;  
}  
/* Output: this is a test  
           4                (count = 4  
because "this" is 4 characters) */
```

FORMAT MODIFIERS FOR printf()

Many format specifiers can take modifiers that alter their meaning. The format modifier goes between the percent sign and the format code.

Modifier Type	Syntax	Description
Minimum field width	% n format	Integer placed between % and format code; pads output with spaces to ensure minimum length. If data is longer, it prints in full.
Zero padding	% 0n format	Prefix 0 before field width specifier to pad with zeros instead of spaces. e.g., %05d pads to 5 digits with leading zeros.
Precision	%n.p format	Period followed by an integer after the field width; meaning depends on data type (see below).

Modifier Type	Syntax	Description
Left justification	<code>%-n format</code>	Minus sign directly after % forces output to be left-justified within the field. Default is right-justified.
Long integer	<code>%ld / %li etc.</code>	l (ell) modifier: tells printf() a long data type follows. Can be used with d, i, o, u, x.
Short integer	<code>%hd / %hu etc.</code>	h modifier: tells printf() to display a short integer. Can be used with d, i, o, u, x.
Long double	<code>%Lf / %Le etc.</code>	L modifier: used with e, f, g to indicate a long double argument.
Wide character (C95)	<code>%lc / %ls</code>	l modifier with c: wide character. l modifier with s: wide-character string.
hh (C99)	<code>%hhd etc.</code>	Applied to d, i, o, u, x, or n. Argument is a signed/unsigned char, or pointer to signed char (for n).
ll (C99)	<code>%lld etc.</code>	Applied to d, i, o, u, x, or n. Argument is a signed/unsigned long long int, or pointer to long long int (for n).
# modifier	<code>##%x, ##%o, ##%f etc.</code>	Before g/G/f/E/e: ensures decimal point even with no decimal digits. Before x/X:

Modifier Type	Syntax	Description
		adds 0x prefix. Before o: adds leading zero.
* modifier	%.*f	Use * as placeholder for width or precision; printf() matches it to the next argument in order.

Minimum Field Width Specifier

An integer between the % sign and the format code acts as the minimum field width. If the output is shorter, it is padded with spaces on the left (right-justified by default). If the output is longer, it prints in full — the minimum is never exceeded downward but can be exceeded upward.

```
#include <stdio.h>
int main(void)
{
    double item = 10.12304;
    printf("%f\n", item);    /* 10.123040
(no padding) */
    printf("%10f\n", item);  /*
10.123040                (10 chars wide, right-
justified) */
    printf("%012f\n", item); /*
00010.123040            (12 chars, zero-
padded) */
    return 0;
}
```

Most commonly used to produce tables with aligned columns:

```
/* Table of squares and cubes, columns 8
chars wide */
for(i=1; i<20; i++)
    printf("%8d %8d %8d\n", i, i*i, i*i*i);
```

Precision Specifier

The precision specifier follows the minimum field width (if present). It is a period followed by an integer. Its meaning depends on the type of data:

Applied to	Meaning of Precision
%f, %e, %E	Number of decimal places displayed. e.g., %10.4f — at least 10 chars wide, 4 decimal places.
%g, %G	Number of significant digits displayed.
%s (strings)	Maximum field length. If string is longer than the maximum, end characters are truncated. e.g., %5.7s — at least 5, not exceeding 7 chars.
Integer types (%d etc.)	Minimum number of digits that will appear. Leading zeroes are added to achieve the required digit count.

Example:

```
printf("%.4f\n",          123.1234567);  /*
Output: 123.1235          (4 decimal places)
*/
printf("%3.8d\n",        1000);          /*
Output: 00001000          (min 8 digits) */
printf("%10.15s\n",      "This is a simple
test.");
/* Output:  This is a simpl
(max 15 chars) */
```

Justifying Output

By default, all output is right-justified within its field. Place a minus sign directly after the % to force left justification.

```
#include <stdio.h>
int main(void)
{
    printf(".....\n");
    printf("right-justified: %8d\n", 100);
/* output:          100 */
    printf(" left-justified: %-8d\n", 100);
/* output: 100      */
    return 0;
}
```

The * and # Modifiers

The # modifier:

- Before g, G, f, E, e — ensures a decimal point is present even if there are no decimal digits

- Before x or X — adds the 0x prefix to the hexadecimal number
- Before o — causes the number to be printed with a leading zero
- In C99, # can also be applied to %a, ensuring a decimal point is displayed
- Cannot be applied to any other format specifiers

The * modifier: Instead of constants, minimum field width and precision can be provided as arguments to printf(). Use * as a placeholder; printf() matches * to the next argument in order.

```
#include <stdio.h>
int main(void)
{
    printf("%x %#x\n", 10, 10);          /*
Output: a 0xa */
    printf("%*.*f", 10, 4, 1234.34);
/* width=10, precision=4 */
    return 0;
}
/* In %*.*f: first * matches 10 (width),
second * matches 4 (precision) */
```

scanf()

scanf() is the general-purpose console input routine. It can read all the built-in data types and automatically converts numbers into the proper internal format. It is much like the reverse of printf().

Prototype:

```
int scanf(const char *control_string, ...);
```

- Returns the number of data items successfully assigned a value
- Returns EOF if an error occurs
- The control_string determines how values are read into the variables pointed to in the argument list

The control string consists of three classifications of characters:

Classification	Description
Format specifiers	Begin with % and tell scanf() what type of data to read next
White-space characters	Space, tab, newline in the control string causes scanf() to skip over one or more leading white-space characters in the input stream
Non-white-space characters	A non-white-space character in the control string causes scanf() to read and discard the matching character from the input stream

scanf() Format Specifiers (Table 8-3)

Code	Meaning
%c	Reads a single character

Code	Meaning
%d	Reads a decimal integer
%i	Reads an integer in decimal, octal, or hexadecimal format
%u	Reads an unsigned decimal integer
%f	Reads a floating-point number
%e	Reads a floating-point number
%g	Reads a floating-point number
%o	Reads an octal number
%x	Reads a hexadecimal number
%s	Reads a string (stops at first white-space character)
%p	Reads a pointer (memory address)
%n	Stores the number of characters read so far into the pointed-to integer; does not consume input
%[]	Scanset — reads characters as long as they match the set defined inside []
%%	Reads a literal percent sign
%a	Reads a floating-point number — C99 only

Inputting Numbers

- **%d** or **%i** — read a signed decimal integer. **%i** also accepts octal (prefix 0) and hexadecimal (prefix 0x) input
- **%f**, **%e**, or **%g** — read a floating-point number in standard or scientific notation. C99 also adds **%a**
- **%o** — read an integer in octal format

- `%x` — read an integer in hexadecimal format; `%x` accepts letters A–F in either upper or lowercase
- `%u` — read an unsigned integer

Example — read octal and hexadecimal numbers:

```
#include <stdio.h>
int main(void)
{
    int i, j;
    scanf("%o%x", &i, &j);
    printf("%o %x", i, j);
    return 0;
}
/* scanf() stops reading a number when the
first non-numeric character is encountered
*/
```

Reading Individual Characters Using `scanf()`

Use the `%c` format specifier to read a single character with `scanf()`. However:

- Like `getchar()`, `scanf()` with `%c` will generally line-buffer input — somewhat troublesome in interactive environments
- White-space characters (spaces, tabs, newlines) ARE read as regular characters when using `%c` — unlike other format specifiers where they act as field separators

```
/* With input stream: "x y" */
scanf("%c%c%c", &a, &b, &c);
```

```
/* a = 'x', b = ' ' (space), c = 'y' */
```

Reading Strings

Use `%s` to read a string with `scanf()`. It reads characters until it encounters a white-space character.

- Characters read are placed into the character array pointed to by the corresponding argument
- The result is null-terminated automatically
- White-space characters for `scanf()` are: space, newline, tab, vertical tab, or formfeed
- Unlike `gets()` (which reads until ENTER), `scanf()` reads until the first white space — you cannot use it to read strings containing spaces

Example — only reads "hello" from input "hello there":

```
#include <stdio.h>
int main(void)
{
    char str[80];
    printf("Enter a string: ");
    scanf("%s", str);
    printf("Here's your string: %s", str);
    return 0;
}
/* If user types "hello there", only
"hello" is stored in str.
The space terminates reading. */
```

NOTE *Strings are read into character arrays, and the array name without any index is already a pointer to the first element. Therefore, do NOT use & before the array name when reading a string with scanf().*

Inputting an Address

Use %p to read a memory address. scanf() reads the address in the format defined by the CPU architecture.

```
#include <stdio.h>
int main(void)
{
    char *p;
    printf("Enter an address: ");
    scanf("%p", &p);
    printf("Value at location %p is %c\n",
p, *p);
    return 0;
}
```

The %n Specifier in scanf()

The %n specifier in scanf() instructs it to store the number of characters read from the input stream (up to the point at which %n was encountered) in the integer variable pointed to by the corresponding argument.

Using a Scanset — %[]

The scanset is a general-purpose format specifier. It defines a set of characters. When scanf() processes a scanset, it reads

characters as long as those characters are part of the defined set. The characters are assigned to the character array pointed to by the scanset's corresponding argument.

How to define a scanset: put the characters to scan for inside square brackets, preceded by %

```
%[XYZ]          /* reads only X, Y, and Z
characters */
%[A-Z]          /* reads characters A through
Z (range using hyphen) */
%[^abc]         /* reads any character EXCEPT
a, b, c (inverted set using ^) */
```

- `scanf()` continues reading characters into the corresponding array until it encounters a character not in the scanset
- The array is null-terminated upon return from `scanf()`
- The scanset is case sensitive — to scan both cases, specify each explicitly
- Most implementations support range notation with a hyphen (e.g., `[A-Z]`)
- The `^` character as the first character in the set instructs `scanf()` to accept any character NOT defined in the scanset

Example:

```
#include <stdio.h>
int main(void)
{
```

```
    int i;
    char str[80], str2[80];
    scanf("%d%[abcdefg]%s", &i, str, str2);
    printf("%d %s %s", i, str, str2);
    return 0;
}
/* Input: 123abcdtye
   Output: 123 abcd tye
   Because 't' is not in the scanset,
   scanf() stops reading into str at 't'.
   The remaining characters 'tye' go into
   str2. */
```

Discarding Unwanted White Space

A white-space character in the control string causes `scanf()` to skip over one or more leading white-space characters in the input stream. One white-space character in the control string causes `scanf()` to read (but not store) any number (including zero) of white-space characters up to the first non-white-space character.

White-space characters include: space, tab, vertical tab, formfeed, and newline.

Non-White-Space Characters in the Control String

A non-white-space character in the control string causes `scanf()` to read and discard the matching character from the input stream. If the specified character is not found, `scanf()` terminates.

```
/* Read two integers separated by a comma
*/ scanf("%d,%d", &x, &y);
/* scanf() reads an integer, reads and
discards the comma, then reads another
integer */

/* To read and discard a percent sign, use
%% in the control string */
```

You Must Pass scanf() Addresses

All variables used to receive values through scanf() must be passed by their addresses (as pointers). This is how C creates a call by reference, which allows a function to alter the contents of an argument.

```
/* Reading a single integer: */
scanf("%d", &count);    /* &count passes the
ADDRESS of count */

/* Reading a string — array name is already
a pointer, so no & needed: */
scanf("%s", str);       /* str is already a
pointer to first element */

/* Reading two integers: */
scanf("%d %d", &x, &y);
```

NOTE *Forgetting the & operator before scalar variable arguments is one of the most common programming errors with scanf(). If you omit &, the*

value of the variable is passed as an address, which will almost certainly cause a runtime error or crash.

Format Modifiers for scanf()

Like printf(), scanf() allows several modifiers for its format specifiers:

Modifier	Syntax	Description
Maximum field width	% n format	Integer between % and format specifier limits the number of characters read for that field. If input stream exceeds the limit, subsequent calls begin where this call left off.
Long integer	%ld, %li etc.	l (ell) in front of d, i, o, u, x, n — reads a long integer
Short integer	%hd, %hu etc.	h in front of d, i, o, u, x, n — reads a short integer
Double (float)	%lf, %le, %lg	l in front of f, e, g — scanf() assigns data to a double (default is float)
Long double	%Lf, %Le, %Lg	L in front of f, e, g — reads a long double
Wide char (C95)	%lc, %ls	l with c: pointer to wchar_t. l with s: pointer to wchar_t array. l can also modify scanfset for wide chars.

Modifier	Syntax	Description
hh (C99)	%hhd etc.	Applied to d, i, o, u, x, n — argument is a pointer to signed/unsigned char
ll (C99)	%lld etc.	Applied to d, i, o, u, x, n — argument is a pointer to signed/unsigned long long int
Assignment suppression	% *format	* before format code: reads the field but does NOT assign it to any variable

Example — maximum field width:

```
/* Read at most 20 characters into str */
scanf("%20s", str);
/* If input is ABCDEFGHIJKLMNOPQRSTUVWXYZ,
   only ABCDEFGHIJKLMNOPQRST (20 chars)
   goes into str.
   The next scanf() call will start from
   UVWXYZ. */

/* Input terminates before max width if
   white space is encountered */
```

Suppressing Input

You can tell `scanf()` to read a field but NOT assign it to any variable by preceding the field's format code with an asterisk (*). This is called assignment suppression.

```
/* Read two integers separated by a comma —
discard the comma */ scanf("%d%c%d", &x,
&y);
/* The %c reads and discards one character
(the comma) without assigning it */

/* Input: 10,10
   x = 10, y = 10, comma is read but
discarded */
```

- Assignment suppression is especially useful when you need to process only part of the entered input
- The `*` modifier applies to any format specifier

CHAPTER 8 — QUICK REFERENCE SUMMARY

Character and String I/O Functions

Function	Prototype	Key Behaviour
<code>getchar()</code>	<code>int getchar(void)</code>	Reads char from keyboard; echoes to screen; line-buffered; returns EOF on error
<code>putchar()</code>	<code>int putchar(int c)</code>	Writes char to screen; returns char written or EOF on error
<code>getch()</code>	<code>int getch(void)</code>	Non-standard; no echo; immediate (not line-buffered); in <code><conio.h></code>

Function	Prototype	Key Behaviour
getche()	int getche(void)	Non-standard; with echo; immediate; in <conio.h>
gets()	char *gets(char *str)	Reads string until ENTER; null-terminates; no boundary check — unsafe for commercial code
puts()	int puts(const char *str)	Writes string + newline; faster than printf(); returns nonneg or EOF

printf() Format Specifiers Quick Reference

Specifier	Type / Output
%c	char — single character
%d / %i	int — signed decimal integer
%u	unsigned int — unsigned decimal
%f	double — decimal floating-point
%e / %E	double — scientific notation (lowercase/uppercase e)
%g / %G	double — shorter of %f or %e (lowercase/uppercase)
%o	unsigned int — octal
%x / %X	unsigned int — hex (lower/uppercase)
%s	char* — null-terminated string
%p	pointer — memory address

Specifier	Type / Output
<code>%n</code>	<code>int*</code> — stores count of chars written so far
<code>%%</code>	literal <code>%</code> sign

printf() Format Modifier Syntax

```
% [flags] [width] [.precision] [length
modifier] type
```

Flags: - (left-justify) 0 (zero-pad)

 # (alt form: 0x prefix for hex,
decimal point for floats)

 * (width or precision taken from
next argument)

Width: integer — minimum field width

Precis: .integer — decimal places / max
string length / min digits

Length: h (short) l (long / double)
L (long double)

 hh (char) ll (long long)

[C99]

Type: c d i u f e E g G o x X s p n %%

scanf() Format Specifiers Quick Reference

Specifier	Reads
<code>%c</code>	single character (white space included)

Specifier	Reads
%d	signed decimal integer
%i	integer: decimal, octal (0 prefix), or hex (0x prefix)
%u	unsigned decimal integer
%f / %e / %g	floating-point number (assigned to float by default)
%lf / %le / %lg	floating-point number assigned to double
%o	octal integer
%x	hexadecimal integer
%s	string — stops at first white space; null-terminated
%p	pointer (memory address)
%n	stores count of chars read so far; no input consumed
%[set]	characters matching the set; null-terminated
%[^set]	characters NOT in the set (inverted scanset)
%%	literal % sign

scanf() Key Rules

Rule	Detail
Always pass addresses	All scalar arguments must be preceded by & operator; array names are already pointers
%s stops at white space	scanf() reads a string until space/tab/newline — cannot read multi-word strings
%c reads white space	When reading a single char with %c, spaces and newlines ARE read as data
Maximum field width	Integer before format code limits characters read for that field
Assignment suppression	* before format code reads but discards — e.g., %*c discards one character
Scanset %[]	Reads only characters in the defined set; stops at first non-matching char
Inverted scanset %[^]	Reads any character NOT in the set
White space in control string	Skips any number of leading white-space characters in input
Non-white-space in control string	Reads and discards the matching character; terminates if not found

— End of Chapter 8 Notes —